Notebook Logger Interpretation & Playbook

Detailed Specification for In-Memory Tracking, Asynchronous Pipelines, and State Transitions

This document bridges the log-writing architecture of the `notebook-edit-tracker` extension with the design requirements of your reproduction and visualization platform. It defines how data streams map onto virtual state boundaries, what information is persisted versus what must be generated in memory, and how to execute deterministic sequential replays.

1. The Log Pipeline & Stream Separation

The logger processes rapid, high-frequency editor events by splitting them cleanly into two highly decoupled append-only log files written as JSON Lines (JSONL). Your playback application must ingest these streams, merge them chronologically, and resolve individual state actions.

Log Stream Type	Log File Suffix	Architectural Intent & Role Division
Structure Changes	`{notebook}_structure_changes.json`	Logs absolute layout boundaries. Tracks layout positions, type configurations, cell executions, languages, insertions, and structural shifts.
Content Changes	`{notebook}_content_changes.json`	Logs atomic, cell-internal character deltas. Dedicated solely to capturing textual mutations occurring within active editor spaces.

> **Optimization Rule: Internal De-duplication Buffer**
> To prevent excessive log sizes, the logging pipeline relies on a ring buffer of size 5. Consecutive logs that yield identical structural metadata signatures (excluding timestamps) are automatically suppressed and muted from disk.

2. In-Memory Tracking Infrastructure

The replay program must act as a precise state interpreter. Because the disk logs provide contextual information only at the precise millisecond an action happens, your engine **must actively maintain two virtual data models in memory** to resolve historical positions and references.

A. The Virtual Structural Array (`notebookLayout`)

- **Definition:** A dynamic, sequential list of strings containing the active `cellUri` values in their exact display order.
- **Why it must be in memory:** Structural log actions only specify a target `cellIndex`. They do not tell you which cells are neighboring it or which cell was present at that position prior to an operation. Maintaining this array allows your engine to know exactly what cell sits at any given index at any timestamp.

B. The Text Buffer Dictionary (`textBuffers`)

- **Definition:** A key-value lookup map (e.g., `Map<string, string>`) linking an immutable `cellUri` directly to a mutable text string block.

> **Mandatory Architectural Boundary: Decoupling Identity from Index**
>
> **Never bind text content directly to structural indices.** Cell positions are unstable and shift rapidly during selections, insertions, moves, and deletions. Text maps must look up data exclusively via the immutable, unique `cellUri` string (extracted from the VS Code cell fragment URI, e.g., `ch4~00002`).

3. The State Machine Routine & Action Rules

Your reproduction program must loop sequentially through the unified chronological event timeline. Each structural or textual entry mutates the in-memory virtual state tracking layouts as defined below:

3.1 Structural Mutations (From `structure_changes.json`)

changeType	Virtual State Machine Operation Details	In-Memory Array Impact
`insert`	Inject the provided `cellUri` into the `notebookLayout` array at index position `cellIndex`. Allocate an empty string entry in your `textBuffers` map for this key if it is absent.	All items originally situated at or past `cellIndex` shift down ($+1$).
`delete`	Locate the element at position `cellIndex` within your virtual `notebookLayout` list and remove it. **Crucial Rule:** Do not erase its record from the `textBuffers` dictionary; keep the code text cached to support potential future Undo operations.	All items originally situated at or past `cellIndex` shift up (-1).
`move`	The log registers the cell's **final destination index** under `cellIndex`. The source index is intentionally omitted from the log entry. **Execution Steps:** 1. Scan your live in-memory `notebookLayout` array to find the current position of the target `cellUri` (this resolves your source index). 2. Remove the cell from that discovered source position. 3. Inject the cell back into the array at the target `cellIndex` specified by the log.	Dynamically shifts intermediate values depending on the directional direction of the move.
`language_change`	Update the cell's format properties (e.g., toggling between Code or Markdown rendering view modes). Does not affect text buffers or cell position order.	No index variation.
`execute`	Bind the `executeOutput` payload onto your visualization layer for this specific cell. The payload preserves cell output formats, embedded text matrices, rich tabular HTML, or base64 data for inline charts.	No index variation.

3.2 Content Mutations (From `content_changes.json`)

When an item from the content log fires, bypass the structural array completely. Go straight to the `textBuffers` map, extract the string buffer corresponding to the log's `cellUri`, and map the `contentChanges` array sequentially:

- **Character Insertions:** Slice and insert the exact literal string defined in `text` right into the buffer at the position designated by the `rangeOffset` integer.
- **Character Removals/Replacements:** Erase a segment of text starting from `rangeOffset` for a length of `rangeLength` characters, then inject any characters stored inside the `text` parameter.

4. Advanced Boundary Handling & Defensive Replay Tips

4.1 The Non-Empty Insertion Routine (Dual-Log Routine)

When a user copies an existing block of cells, utilizes an AI generation prompt, or triggers an Undo instruction that resurrects text, the layout engine applies a synchronized dual-logging design strategy. It registers an `insert` event in the structural log and, at the **exact same millisecond timestamp**, writes a mock text payload containing the full initial block content into the content log file.

> **Required Sorting Tiebreak Sequence**
>
> Your replay processing algorithm must execute a strict **Global Chronological Sort** across both files using the `timeStamp` value. In cases where timestamps are perfectly identical, your loop must evaluate **Structural changes before Content changes**. This guarantees that your virtual `notebookLayout` array expands to create the dynamic cell identity space before the text mutation routine attempts to manipulate characters within its buffer offset.

4.2 Multi-Cell Move Stream Matching

VS Code handles cell moves as an atomic compound operation. In the structure log, this surfaces sequentially as a `delete` operation at the original index, followed immediately by an insertion line bearing the `move` action token. This is why you must retain the deleted cell's text inside the `textBuffers` map instead of purging it—when the subsequent `move` action executes a moment later, the cell retains its text history perfectly.

4.3 Runtime Output Debounce Management

Rapid execution steps can log overlapping intermediate states. The tracking extension enforces a $1000 ext{ms}$ execution filter to drop identical consecutive outputs. In your visualization software, ensure that once an `executeOutput` array is captured, that data layer remains active and rendered until a structural layout deletion or a subsequent execution frame overrides it.

5. Sequential Replay Engine Roadmap

Ensure your reproduction implementation codebase adheres to this linear execution flow:

1. **Global Log Parsing:** Read both JSONL streams and fully deserialize every line into a single collective array list.
2. **Chronological Sort:** Sort the array list in ascending order by `timeStamp`. For identical timestamps, enforce that structural entries are placed before text changes.
3. **Virtual State Instantiation:** Set up an empty tracking layout list (`string[]`) and an empty lookup map object for string contents.
4. **State Machine Playback Loop:** Step linearly through the sorted timeline array. Feed each log line through the action routines to dynamically alter the structural array and character buffers.
5. **Decoupled Presentation View:** Separate your state mutation code from your UI rendering functions. Have your user interface look directly at the in-memory models (`notebookLayout` and `textBuffers`) to drive layout transitions, step counters, sliders, and animation speeds.